

আপনার দাবা অ্যাপ্লিকেশনের (Next-chess) জন্য তৈরি করা এই টাইপ আর্কিটেকচারটি অত্যন্ত চমৎকার এবং পরিপক্ব (mature)। একটি WebAssembly Stockfish Engine-কে ব্রাউজার ব্যাকগ্রাউন্ডে (Web Worker-এর মাধ্যমে) স্মুথলি এবং কোনো প্রকার রেস-কন্ডিশন (race condition) ছাড়া রান করানোর জন্য এই প্যাটার্নটি একদম আইডিয়াল।

নিচে পুরো বিষয়টিকে এবং আমি যে পরিবর্তনগুলো সাজেস্ট করেছি, সেগুলো বাংলায় গভীরভাবে ব্যাখ্যা করা হলো:

১. কোর কনসেপ্ট: EngineSearchPurpose ও EngineSearchContext

দাবা অ্যাপে ইঞ্জিনকে বিভিন্ন কারণে সার্চ কমান্ড পাঠাতে হয়। আপনি সেটিকে 'ai', 'hint', এবং 'eval'-এ ভাগ করে দারুণ কাজ করেছেন।

- **কেন এটি গুরুত্বপূর্ণ?**
ধরে নিন, ব্যাকগ্রাউন্ডে ক্রমাগত 'eval' (পজিশন অ্যানালাইসিস) চলছে। ঠিক ওই মুহুর্তে ইউজার একটা চাল চালল এবং এআই নিজের চাল খোঁজার জন্য ইঞ্জিনের কাছে রিকোয়েস্ট পাঠাল।
- **সমস্যা (Race Condition):** ইঞ্জিন যখন রেসপন্স ব্যাক করবে, তখন আপনার মেইন থ্রেড (UI) কীভাবে বুঝবে যে এই bestmove বা evaluation রেজাল্টটি ব্যাকগ্রাউন্ড অ্যানালাইসিসের নাকি এআই প্লেয়ারের?
- **সমাধান (requestId ও purpose):** প্রতিটি রিকোয়েস্টের সাথে একটি ইউনিক requestId এবং তার উদ্দেশ্য (purpose) জুড়ে দিয়ে আপনি নিশ্চিত করেছেন যে, মেইন থ্রেড আসা মাত্রই বুঝতে পারবে এই ডেটা কোথায় রেন্ডার বা প্রসেস করতে হবে। একেই বলে **Contextualized Messaging Pattern**।

২. টাইপ রিফাইনমেন্টের গভীর ব্যাখ্যা (Deep Architectural Breakdown)

আমি আপনার টাইপগুলোতে যে সামান্য কিছু পরিবর্তন এনেছি, সেগুলো করার মূল কারণ নিচে ভেঙে বলা হলো:

ক) অসম্ভব স্টেট রোধ করা (EngineScore বনাম null ভ্যালু)

আপনার মেইন টাইপে EngineHintEvent-এর ভেতর এমন ছিল:

```
TypeScript
  value: number | null
  mate: number | null
```

সমস্যা: টাইপস্ক্রিপ্টের নিয়ম অনুযায়ী এখানে একটি লজিক্যাল বাগ বা **Impossible State** তৈরি হওয়ার সুযোগ থাকে। যেমন, কোনো কারণে একই সাথে দুটিই number হয়ে গেল, অথবা দুটিই null হয়ে গেল। কিন্তু বাস্তব দাবার ইঞ্জিনে (যেমন UCI প্রোটোকলে) একটি পজিশন হয় সেন্টিপন (cp) দিয়ে মূল্যায়ন হবে (যেমন: +১.৫ বা ১৫০), অথবা সেটি সরাসরি কিস্তিমাত বা মেট (mate) এর দিকে যাবে (যেমন: ৩ চালে মেট)।

সমাধান: আমি এটিকে একটি **Discriminated Union**-এ রূপান্তর করেছি:

```
TypeScript
export type EngineScore =
  type 'cp' value number // Centipawns (যেমন: +1.5 হলে 150)
  type 'mate' value number // Forced Mate (যেমন: ৩ চালে কিস্তিমাত হলে 3)
```

এর ফলে আপনার কোডে যখন `score.type === 'cp'` হবে, তখন টাইপস্ক্রিপ্ট স্বয়ংক্রিয়ভাবে জানবে যে এর ভেতর মেটের কোনো হিসাব নেই। এতে UI-তে কন্ডিশনাল রেন্ডারিং করা অনেক সহজ এবং শতভাগ নিরাপদ হবে।

খ) EngineReadyEvent এবং EngineEvent ইউনিয়ন সেফটি

আপনার আগের EngineEvent ইউনিয়নে EngineReadyEvent এর সাথে EngineSearchContext যুক্ত ছিল না (যা একদম সঠিক, কারণ ইঞ্জিন যখন রেডি হয় তখন কোনো স্পেসিফিক রিকোয়েস্ট আইডি থাকে না)। কিন্তু অন্য ইভেন্টগুলো নেস্টেড ব্রাকেটের ভেতর ছিল।

টাইপস্ক্রিপ্ট যেন খুব সহজে এবং দ্রুত কম্পাইল টাইমে টাইপ গার্ড (switch case বা if condition) ডিটেক্ট করতে পারে, সেজন্য পুরো ইউনিয়নটিকে এভাবে সাজানো হয়েছে:

```
TypeScript
export type EngineEvent =
  EngineReadyEvent
  EngineErrorEvent
  EngineSearchContext & EngineBestMoveEvent
  EngineSearchContext & EnginePonderEvent
```

এখানে মেইন থ্রেড (UI) যখনই কোনো মেসেজ পাবে, সে সরাসরি `event.type` চেক করেই সিদ্ধান্ত নিতে পারবে। যেমন:

- `event.type === 'ready'` হলে লোডার বন্ধ হবে।
- `event.type === 'bestmove'` হলে সে নিশ্চিতভাবেই `requestId` এবং `purpose` অ্যাক্সেস করতে পারবে, কারণ টাইপস্ক্রিপ্ট ইন্টারসেকশন (&) টাইপকে ফোর্স করবে।

গ) এরর হ্যান্ডলিং ও পন্ডার মুভ (EngineErrorEvent & ponder)

- **EngineErrorEvent:** আপনার স্টকফিশ ১৮ ইঞ্জিনটি যেহেতু WASM ফাইল (যা প্রায় ১০০ মেগাবাইটের

উপরে হতে পারে এবং ভারী মেমোরি নেয়), ব্রাউজারে অনেক সময় মেমোরি ক্র্যাশ বা ফাইল লোডিং এরর হতে পারে। তাই ইভেন্ট স্ট্রাকচারে একটি অফিশিয়াল error টাইপ রাখা অত্যন্ত জরুরি, যাতে অ্যাপ ক্র্যাশ না করে ইউজারকে সুন্দর মেসেজ দেখানো যায়।

- **ponder:** স্টকফিশ যখন কোনো বেস্ট মুভ (bestmove) দেয়, সে সাথে প্রায়ই একটি ponder মুভও দেয় (অর্থাৎ, "আমার চাল তো e2e4, তবে আমার ধারণা অপোনেন্ট এরপর e7e5 চালবে")। আপনি যদি ভবিষ্যতে আপনার ইঞ্জিনের পারফরম্যান্স বাড়াতে চান, তবে এই ponder চালটি দিয়ে অপোনেন্টের চালের আগেই ব্যাকগ্রাউন্ডে ক্যালকুলেশন শুরু করে দিতে পারবেন।



আপনার প্রজেক্টের জন্য আর্কিটেকচারাল গাইডলাইন

আপনার Web Worker-এর ভেতর কোড লেখার সময় এই টাইপটি যেভাবে কাজ করবে:

1. **Worker State:** ওয়ার্কারের ভেতরে একটি গ্লোবাল অবজেক্ট রাখুন, যেমন: `let currentContext: EngineSearchContext | null = null;`
2. **Request Reception:** মেইন থ্রেড থেকে যখন `EngineSearchRequest` আসবে, তখন ওয়ার্কার `currentContext = { requestId, purpose }` সেট করে স্টকফিশকে বলবে `go depth 18।`
3. **Parsing Line:** স্টকফিশ (WASM) থেকে প্রতি লাইনে যে টেক্সট আউটপুট আসবে, ওয়ার্কার সেটিকে পার্স করে `ParsedEngineEvent` (যেমন `evaluation` বা `bestmove`) এ রূপান্তর করবে।
4. **Posting Back:** মেইন থ্রেডে `postMessage` করার ঠিক আগে ওয়ার্কার কোডটি হবে অনেকটা এমন:

TypeScript

```
// Worker-এর ভেতরের লজিক (কাল্পনিক উদাহরণ)
if (parsedEvent.type === 'ready' & currentContext) {
  const finalEvent: EngineEvent =
    ...currentContext
    ...parsedEvent
  // ...
  self.postMessage(finalEvent)
} else {
  self.postMessage(parsedEvent) // ready বা error এর জন্য
}
```

এই আর্কিটেকচারটি ফলো করলে আপনার Next-chess অ্যাপ্লিকেশনের স্টেট ম্যানেজমেন্ট (State Management) অত্যন্ত রিজিড, প্রিডিক্টেবল এবং বাগ-ফ্রি থাকবে।